SA UVM Guide

An annotated, source-complete walkthrough of the SA serial adder verification environment

Project baseline: sa_VHDL_UVM_all_transaction_methods.zip | RTL: VHDL-2008 | Testbench: SystemVerilog / UVM | Questa mixed-language workflow

How to use this guide

Read Chapters 1–5 sequentially to understand the testbench stimulus chain. Chapters 6–9 explain the observers and verification infrastructure. Appendix A reproduces every textual source file from the supplied project for cross-reference. No simulator execution is claimed.

Contents

1. Understanding the SA DUT

2. Transaction: the unit of stimulus

2A. Seven transaction methods and practical test

3. Sequence and sequencer

4. Driver: translating transactions into clocked pins

5. Complete stimulus handshake

6. Monitors and the prediction path

7. Scoreboard and test control

8. Configuration, factory and environment

9. Compiling and debugging in Questa

Appendix A. Full source listings

References and verification checklist

1. Understanding the SA DUT

The SA design is a serial adder. Each request consists of two 2-bit operands (ina and inb). Its VHDL ports are single-bit ina and inb because the operands are transmitted serially, most significant bit first. A three-bit sum is emitted serially as result_o and qualified by en_o.

Signal-level connection: sa_sequence → sa_sequencer → sa_driver → sa_if → sa (VHDL DUT); sa input/output monitors → expected/actual analysis streams → sa_scoreboard.

- clk: DUT rising-edge sample clock; driver prepares values at falling edges.
- en_i: request initiation; asserted during first operand bit in the supplied driver.
- ina/inb: serial operand pins; bit[1] is presented before bit[0].
- result_o: output result pin, replacing the reserved VHDL identifier out.
- en_o: output qualification pulse; monitor behavior determines exactly which cycles become transactions.

2. Transaction: the unit of stimulus

A UVM transaction is a SystemVerilog class that represents one meaningful transfer independently of pin-level timing. sa_transaction extends uvm_sequence_item. rand bit[1:0] ina and inb are randomized operand values; bit[2:0] out holds the sum. The transaction does not control en_i or clock edges: those belong to the driver.

The uvm_object_utils macro registers this type with the UVM factory. type_id::create allows future factory overrides. new constructs the object. The new revision implements convert2string, do_copy, do_compare, do_print, do_record, do_pack, and do_unpack; all are exercised from the testbench. In this source, do_compare compares all three fields, although the scoreboard intentionally compares the result field only.

sa_transaction.svh

Study the class declaration and methods below. A transaction object can carry both generated inputs and observed/predicted result values, but each producer must populate the fields it owns.

Complete source: sa_transaction.svh (78 lines)

```
`ifndef __SA_TRANSACTION_SVH__
`define __SA_TRANSACTION_SVH__
// One 2-bit input pair and its 3-bit sum; explicit UVM field hooks.
class sa_transaction extends uvm_sequence_item;
  `uvm_object_utils(sa_transaction)
  rand bit [1:0] ina;
  rand bit [1:0] inb;
       bit [2:0] out;

  function new(string name = "sa_transaction");
    super.new(name);
  endfunction

  function string convert2string();
    return $sformatf("ina=%0d (0x%0h) inb=%0d (0x%0h) out=%0d (0x%0h)",
                     ina, ina, inb, inb, out, out);
  endfunction

  // Invoked via destination.copy(source).
  function void do_copy(uvm_object rhs);
    sa_transaction src;
    if (!$cast(src, rhs))
      `uvm_fatal("SA_COPY_TYPE", "rhs is not an sa_transaction")
    super.do_copy(rhs);
    ina = src.ina;
    inb = src.inb;
    out = src.out;
  endfunction

  // Invoked via expected.compare(actual, comparer).
  function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    sa_transaction other;
    bit matches;
    if (!$cast(other, rhs)) begin
      `uvm_error("SA_COMPARE_TYPE", "rhs is not an sa_transaction")
      return 0;
    end
    matches = super.do_compare(rhs, comparer);
    matches &= comparer.compare_field("ina", ina, other.ina, 2, UVM_DEC);
    matches &= comparer.compare_field("inb", inb, other.inb, 2, UVM_DEC);
    matches &= comparer.compare_field("out", out, other.out, 3, UVM_DEC);
    return matches;
  endfunction

  // Invoked via tx.print() / tx.sprint().
  function void do_print(uvm_printer printer);
    super.do_print(printer);
    printer.print_field_int("ina", ina, 2, UVM_DEC);
    printer.print_field_int("inb", inb, 2, UVM_DEC);
    printer.print_field_int("out", out, 3, UVM_DEC);
  endfunction
```

```
  // Invoked via tx.record(recorder), normally after a stream is opened.
  function void do_record(uvm_recorder recorder);
    super.do_record(recorder);
    recorder.record_field("ina", ina, 2, UVM_DEC);
    recorder.record_field("inb", inb, 2, UVM_DEC);
    recorder.record_field("out", out, 3, UVM_DEC);
  endfunction

  // Invoked via pack_bits(), pack_bytes(), or pack_ints().
  // Width/order MUST match do_unpack: 2 + 2 + 3 = 7 bits.
  function void do_pack(uvm_packer packer);
    super.do_pack(packer);
    packer.pack_field_int(ina, 2);
    packer.pack_field_int(inb, 2);
    packer.pack_field_int(out, 3);
  endfunction

  // Invoked via unpack_bits(), unpack_bytes(), or unpack_ints().
  function void do_unpack(uvm_packer packer);
    super.do_unpack(packer);
    ina = packer.unpack_field_int(2);
    inb = packer.unpack_field_int(2);
    out = packer.unpack_field_int(3);
  endfunction
endclass : sa_transaction
`endif
```

2A. Seven transaction methods: implementation and actual exercise

This updated revision corresponds to sa_VHDL_UVM_all_transaction_methods.zip. The UVM user-facing APIs call virtual hook methods on sa_transaction. Overriding the hook defines field semantics; invoking the public API from the test or scoreboard causes the override to execute. This distinction is important: do_copy is not the same API as copy().

Transaction hook	Called through	What happens
convert2string()	tx.convert2string()	Return an operand/output summary
do_copy(rhs)	dst.copy(src)	Transfer fields after safe type cast
do_compare(rhs, comparer)	lhs.compare(rhs, cmp)	Check corresponding transaction fields
do_print(printer)	tx.sprint()	Print via UVM printer framework
do_record(recorder)	tx.record(rec)	Record fields; database setup is simulator-specific
do_pack(packer)	tx.pack_bits(bits)	Serialize 2+2+3 = 7 bits
do_unpack(packer)	tx.unpack_bits(bits)	Deserialize the seven bits

2A.1 The transaction class: override contracts

In sa_transaction.svh, every method calls its parent where appropriate and processes ina (2 bits), inb (2 bits), and out (3 bits). do_copy uses $cast to verify rhs is a compatible transaction. do_compare must return a bit result; comparisons should use the passed comparer for consistent reporting. The packer/unpacker must use the same field order and widths.

```
`ifndef __SA_TRANSACTION_SVH__
`define __SA_TRANSACTION_SVH__
// One 2-bit input pair and its 3-bit sum; explicit UVM field hooks.
class sa_transaction extends uvm_sequence_item;
  `uvm_object_utils(sa_transaction)
  rand bit [1:0] ina;
```

```
  rand bit [1:0] inb;
       bit [2:0] out;

  function new(string name = "sa_transaction");
    super.new(name);
  endfunction

  function string convert2string();
    return $sformatf("ina=%0d (0x%0h) inb=%0d (0x%0h) out=%0d (0x%0h)",
                     ina, ina, inb, inb, out, out);
  endfunction

  // Invoked via destination.copy(source).
  function void do_copy(uvm_object rhs);
    sa_transaction src;
    if (!$cast(src, rhs))
      `uvm_fatal("SA_COPY_TYPE", "rhs is not an sa_transaction")
    super.do_copy(rhs);
    ina = src.ina;
    inb = src.inb;
    out = src.out;
  endfunction

  // Invoked via expected.compare(actual, comparer).
  function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    sa_transaction other;
    bit matches;
    if (!$cast(other, rhs)) begin
      `uvm_error("SA_COMPARE_TYPE", "rhs is not an sa_transaction")
      return 0;
    end
    matches = super.do_compare(rhs, comparer);
    matches &= comparer.compare_field("ina", ina, other.ina, 2, UVM_DEC);
    matches &= comparer.compare_field("inb", inb, other.inb, 2, UVM_DEC);
    matches &= comparer.compare_field("out", out, other.out, 3, UVM_DEC);
    return matches;
  endfunction

  // Invoked via tx.print() / tx.sprint().
  function void do_print(uvm_printer printer);
    super.do_print(printer);
    printer.print_field_int("ina", ina, 2, UVM_DEC);
    printer.print_field_int("inb", inb, 2, UVM_DEC);
    printer.print_field_int("out", out, 3, UVM_DEC);
  endfunction

  // Invoked via tx.record(recorder), normally after a stream is opened.
  function void do_record(uvm_recorder recorder);
    super.do_record(recorder);
    recorder.record_field("ina", ina, 2, UVM_DEC);
    recorder.record_field("inb", inb, 2, UVM_DEC);
    recorder.record_field("out", out, 3, UVM_DEC);
  endfunction

  // Invoked via pack_bits(), pack_bytes(), or pack_ints().
  // Width/order MUST match do_unpack: 2 + 2 + 3 = 7 bits.
  function void do_pack(uvm_packer packer);
    super.do_pack(packer);
    packer.pack_field_int(ina, 2);
    packer.pack_field_int(inb, 2);
    packer.pack_field_int(out, 3);
  endfunction

  // Invoked via unpack_bits(), unpack_bytes(), or unpack_ints().
  function void do_unpack(uvm_packer packer);
    super.do_unpack(packer);
    ina = packer.unpack_field_int(2);
    inb = packer.unpack_field_int(2);
    out = packer.unpack_field_int(3);
  endfunction
endclass : sa_transaction
```

```
`endif
```

2A.2 How the test exercises the public APIs

sa_test.exercise_transaction_methods() runs before the random DUT sequence. It constructs original, copied, decoded and deliberately changed objects. The following is the exact source of the test method:

```
  task exercise_transaction_methods();
    sa_transaction original, copied, decoded, changed;
    uvm_comparer cmp;
    uvm_recorder rec;
    bit packed_bits[];
    int nbits;

    original = sa_transaction::type_id::create("original");
    copied   = sa_transaction::type_id::create("copied");
    decoded  = sa_transaction::type_id::create("decoded");
    changed  = sa_transaction::type_id::create("changed");
    original.ina = 2'd2;
    original.inb = 2'd3;
    original.out = 3'd5;

    // 1. convert2string()
    `uvm_info("SA_METHOD_STRING", original.convert2string(), UVM_LOW)

    // 2. do_copy() through copy()
    copied.copy(original);
    // 3. do_compare() through compare()
    cmp = new();
    if (!original.compare(copied, cmp))
      `uvm_fatal("SA_METHOD_COPY", "copy/compare failed")

    // 4. do_print() through sprint()/print()
    `uvm_info("SA_METHOD_PRINT", original.sprint(), UVM_LOW)

    // 5. do_record() through record() with a recorder.
    // Backend-specific waveform/database output needs the simulator's
    // transaction-recording configuration.
    rec = new("sa_method_recorder");
    original.record(rec);
    `uvm_info("SA_METHOD_RECORD", "Called record(rec): do_record executed", UVM_LOW)

    // 6. do_pack() and 7. do_unpack() through the bit-array APIs.
    nbits = original.pack_bits(packed_bits);
    if (nbits != 7)
      `uvm_fatal("SA_METHOD_PACK", $sformatf("Expected 7 packed bits; got %0d", nbits))
    void'(decoded.unpack_bits(packed_bits));
    cmp = new();
    if (!original.compare(decoded, cmp))
      `uvm_fatal("SA_METHOD_UNPACK", "pack/unpack round-trip failed")

    // Negative compare demonstrates field-by-field mismatch diagnostics.
    changed.copy(original);
    changed.out = original.out ^ 3'b001;
    cmp = new();
    cmp.show_max = 0; // Suppress intentional mismatch diagnostic output.
    if (original.compare(changed, cmp))
      `uvm_fatal("SA_METHOD_NEGATIVE", "Deliberately changed output compared equal")
    `uvm_info("SA_METHODS_PASS", "All seven UVM transaction methods exercised", UVM_NONE)
  endtask
```

2A.3 Expected checks and debugging

- Copy and equality: copied.copy(original) followed by original.compare(copied,cmp) must pass.
- Print and logging: sprint() returns detailed printer output while convert2string() returns the compact custom representation.
- Recording: record(rec) exercises the hook, but a visible transaction database requires appropriate Questa/UVM recording setup.

- Round trip: pack_bits() must report seven bits and unpack_bits() must reproduce original fields before compare().
- Negative comparison: changed.out intentionally differs; a false comparison is success for the negative test.
- The scoreboard also uses copy() and compare() for actual-vs-expected normalization. This assumes input/output transactions are paired in order.
- The test emits SA_METHODS_PASS when method checks complete, while ADDER_SUMMARY reports normal DUT results.

2A.4 Where to inspect in the source

Read sa_transaction.svh for all implementations, sa_test.sv for explicit method invocations, sa_monitor.sv for convert2string() during observation, and sa_scoreboard.sv for copy/compare during checking. These files are reproduced at their regular chapter locations. README_UVM_METHODS.md summarizes the usage and validation limitations.

2B. Illustrated examples: five UVM transaction hooks

This tutorial expands Chapter 2A using the existing sa_transaction.svh and sa_test.sv implementations. The example transaction is ina = 2 (2'b10), inb = 3 (2'b11), out = 5 (3'b101). These object methods do not directly drive DUT pins; the driver and monitors handle DUT timing.

2B.1 Public UVM APIs versus hook implementations

sa_test public API	sa_transaction override	What it does
copied.copy(original)	do_copy(rhs)	Copy ina, inb and out
original.compare(copied, cmp)	do_compare(rhs, comparer)	Check three fields
original.record(rec)	do_record(recorder)	Offer fields to transaction recorder
original.pack_bits(bits)	do_pack(packer)	Write 2 + 2 + 3 = 7 bits
decoded.unpack_bits(bits)	do_unpack(packer)	Read the same seven bits

The public API invokes the virtual hook. For example, use copied.copy(original), not copied.do_copy(original), in normal test code.

2B.2 do_copy(): copy source fields into destination

```
function void do_copy(uvm_object rhs);
  sa_transaction src;
  if (!$cast(src, rhs))
    `uvm_fatal("SA_COPY_TYPE", "rhs is not an sa_transaction")
  super.do_copy(rhs);
  ina = src.ina;
  inb = src.inb;
  out = src.out;
endfunction


original.ina = 2'd2;
original.inb = 2'd3;
original.out = 3'd5;
copied.copy(original); // invokes copied.do_copy(original)
```

Before copying, copied may hold its default zeros. After copying: copied.ina = 2, copied.inb = 3, copied.out = 5. $cast verifies the rhs object type; the three assignments transfer the fields. Because these fields are scalars, changing copied.out later does not change original.out.

2B.3 do_compare(): field-by-field equality and negative test

```
function bit do_compare(uvm_object rhs, uvm_comparer comparer);
  sa_transaction other;
  bit matches;
  if (!$cast(other, rhs)) begin
    `uvm_error("SA_COMPARE_TYPE", "rhs is not an sa_transaction")
    return 0;
  end
  matches = super.do_compare(rhs, comparer);
  matches &= comparer.compare_field("ina", ina, other.ina, 2, UVM_DEC);
  matches &= comparer.compare_field("inb", inb, other.inb, 2, UVM_DEC);
  matches &= comparer.compare_field("out", out, other.out, 3, UVM_DEC);
  return matches;
endfunction


cmp = new();
if (!original.compare(copied, cmp))
  `uvm_fatal("SA_METHOD_COPY", "copy/compare failed")

changed.copy(original);
changed.out = original.out ^ 3'b001; // 101 -> 100
cmp = new();
cmp.show_max = 0; // suppress expected mismatch reporting
if (original.compare(changed, cmp))
  `uvm_fatal("SA_METHOD_NEGATIVE", "Unexpected equality")
```

Field	original	copied	changed
ina	2	2	2
inb	3	3	3
out	5 (101)	5 (101)	4 (100)
compare() result	—	1 / equal	0 / unequal

The negative test succeeds when compare() returns zero, confirming that a single-bit output difference is detected. comparer.compare_field uses each declared field width and the supplied uvm_comparer for diagnostics.

2B.4 do_record(): send transaction fields to a recorder

```
function void do_record(uvm_recorder recorder);
  super.do_record(recorder);
  recorder.record_field("ina", ina, 2, UVM_DEC);
  recorder.record_field("inb", inb, 2, UVM_DEC);
  recorder.record_field("out", out, 3, UVM_DEC);
endfunction


rec = new("sa_method_recorder");
original.record(rec); // dispatches to do_record(rec)
```

Conceptual recorded values: ina = 2 (2 bits), inb = 3 (2 bits), out = 5 (3 bits). record() executes the hook, but a visible Questa transaction database requires compatible recorder and simulator recording configuration. A newly created base recorder alone does not establish a waveform database.

2B.5 do_pack(): serialize seven transaction bits

```
function void do_pack(uvm_packer packer);
  super.do_pack(packer);
  packer.pack_field_int(ina, 2);
```

```
  packer.pack_field_int(inb, 2);
  packer.pack_field_int(out, 3);
endfunction


bit packed_bits[];
int nbits;
nbits = original.pack_bits(packed_bits);
if (nbits != 7)
  `uvm_fatal("SA_METHOD_PACK", "Expected seven packed bits")
```

Field layout: ina[1:0] = 10 | inb[1:0] = 11 | out[2:0] = 101. Conceptually the fields concatenate into 1011101 (seven bits). Actual packed-array indexing and individual bit orientation follow the UVM packer configuration. This object serialization is separate from the SA DUT serial clock protocol.

2B.6 do_unpack(): reconstruct the transaction

```
function void do_unpack(uvm_packer packer);
  super.do_unpack(packer);
  ina = packer.unpack_field_int(2);
  inb = packer.unpack_field_int(2);
  out = packer.unpack_field_int(3);
endfunction


void'(decoded.unpack_bits(packed_bits));
cmp = new();
if (!original.compare(decoded, cmp))
  `uvm_fatal("SA_METHOD_UNPACK", "Pack/unpack failed")
```

The order and bit widths MUST match do_pack(): first ina (2), then inb (2), then out (3). The round-trip check verifies decoded.ina = 2, decoded.inb = 3, decoded.out = 5. Reversing the operand unpack order can corrupt the decoded transaction.

2B.7 Complete exercise_transaction_methods() example

```
task exercise_transaction_methods();
  sa_transaction original, copied, decoded, changed;
  uvm_comparer cmp;
  uvm_recorder rec;
  bit packed_bits[];
  int nbits;

  original = sa_transaction::type_id::create("original");
  copied   = sa_transaction::type_id::create("copied");
  decoded  = sa_transaction::type_id::create("decoded");
  changed  = sa_transaction::type_id::create("changed");
  original.ina = 2'd2;
  original.inb = 2'd3;
  original.out = 3'd5;

  copied.copy(original);                  // do_copy
  cmp = new();
  if (!original.compare(copied, cmp))     // do_compare
    `uvm_fatal("COPY", "Copy comparison failed")

  rec = new("sa_method_recorder");
  original.record(rec);                  // do_record

  nbits = original.pack_bits(packed_bits); // do_pack
  if (nbits != 7)
```

```
    `uvm_fatal("PACK", "Expected seven bits")

  void'(decoded.unpack_bits(packed_bits)); // do_unpack
  cmp = new();
  if (!original.compare(decoded, cmp))
    `uvm_fatal("UNPACK", "Round-trip failed")

  changed.copy(original);
  changed.out = original.out ^ 3'b001;
  cmp = new();
  cmp.show_max = 0;
  if (original.compare(changed, cmp))
    `uvm_fatal("NEGATIVE", "Mismatch not detected")

  `uvm_info("SA_METHODS_PASS", "Transaction methods checked", UVM_NONE)
endtask
```

Call this task from sa_test.run_phase() before starting the randomized SA sequence, as already documented in Chapter 2A. This is an explanatory listing; the source listings in the rest of this guide are preserved unchanged.

2B.8 Relationship to scoreboard and UVMF conversion

In the SA UVM scoreboard, the output monitor observes only the result. The scoreboard copies the actual transaction, normalizes operand fields using the corresponding expected transaction, then compares all three fields. In a UVMF split-agent implementation, input and output transaction types may differ; the predictor/scoreboard must compare appropriately matched fields rather than assume a common object layout.

Recommended learning sequence: Create → Initialize → Copy → Compare → Record → Pack → Unpack → Compare → Validate. Simulator execution is not claimed by this explanatory chapter.

3. Sequence and sequencer

A sequence selects and orders stimulus. sa_sequence extends uvm_sequence#(sa_transaction) and generates 15 requests by default. Each loop constructs a fresh item, calls start_item, randomizes it, and calls finish_item. The sequencer arbitrates sequence items and exposes the sequence-item export consumed by the driver.

The typedef sa_sequencer = uvm_sequencer#(sa_transaction) uses the stock parameterized sequencer; a bespoke class is unnecessary for simple arbitration.

sa_sequencer.sv

Note the distinction: sa_sequence is a UVM object that executes body(), while sa_sequencer is a UVM component used as a communication bridge. The test starts the sequence on the agent sequencer.

Complete source: sa_sequencer.sv (24 lines)

```
class sa_sequence extends uvm_sequence#(sa_transaction);
 `uvm_object_utils(sa_sequence)
 extern function   new(string name = "");
 extern task    body();
endclass: sa_sequence
typedef uvm_sequencer#(sa_transaction) sa_sequencer;
function sa_sequence::new(string name = "");
 super.new(name);
endfunction: new
task sa_sequence::body();
 sa_transaction sa_tx;
 if (starting_phase != null)
     starting_phase.raise_objection(this);
 repeat(15) begin
  sa_tx = sa_transaction::type_id::create(.name("sa_tx") ,
               .contxt(get_full_name()) );
  start_item(sa_tx);     /* handshake with driver::get function */
  if( !sa_tx.randomize() )
```

```
   `uvm_error("", "failed to randomize...");
  finish_item(sa_tx);    /* handshake with driver::get function */
 end
 if (starting_phase != null)
  starting_phase.drop_objection(this);
endtask: body
```

4. Driver: translating transactions into clocked pins

sa_driver extends uvm_driver#(sa_transaction), obtains a virtual sa_if via its configuration object, and repeatedly asks the sequencer for the next transaction. Its job is stimulus generation only, not result checking.

Clock discipline: the DUT samples on rising edges. The driver changes pins on falling edges to reduce active-region races. On cycle 0 it asserts en_i and presents each operand MSB; on cycle 1 it presents each LSB. Remaining cycles hold inactive operand pins while the serial result is produced. After the sixth cycle it acknowledges completion with item_done().

sa_driver.sv

Key APIs: seq_item_port.get_next_item(req) blocks until an item is available; @(negedge m_vif.sig_clock) aligns stimulus before the DUT sampling edge; item_done() completes the request handshake. Null interface or missing configuration produce uvm_fatal.

Complete source: sa_driver.sv (34 lines)

```
class sa_driver extends uvm_driver #(sa_transaction);
  `uvm_component_utils(sa_driver)
  virtual sa_if m_vif;
  clk_data_config drv_config;
  function new(string name, uvm_component parent); super.new(name,parent); endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(clk_data_config)::get(this,"","config",drv_config))
      `uvm_fatal("DRIVER_CFG","Missing clk_data_config")
    m_vif = drv_config.m_vif;
    if (m_vif == null) `uvm_fatal("DRIVER_VIF","Null virtual interface")
  endfunction
  task run_phase(uvm_phase phase);
    sa_transaction req;
    m_vif.sig_en_i = 0;
    m_vif.sig_ina = 0;
    m_vif.sig_inb = 0;
    forever begin
      seq_item_port.get_next_item(req);
      // DUT consumes at rising edges; present stimulus at falling edges.
      for (int cycle=0; cycle<6; cycle++) begin
        @(negedge m_vif.sig_clock);
        m_vif.sig_en_i <= (cycle == 0);
        m_vif.sig_ina  <= (cycle == 0) ? req.ina[1] :
                          (cycle == 1) ? req.ina[0] : 1'b0;
        m_vif.sig_inb  <= (cycle == 0) ? req.inb[1] :
                          (cycle == 1) ? req.inb[0] : 1'b0;
      end
      // At posedge after cycle 5 the RTL completes its six-cycle operation.
      @(posedge m_vif.sig_clock);
      seq_item_port.item_done();
    end
  endtask
endclass
```

5. Complete stimulus handshake

Sequence calls start_item → sequencer grants transaction → sequence randomizes and calls finish_item → driver get_next_item → driver presents pins on falling edges → VHDL DUT captures pins on rising edges → driver item_done → sequence may advance. The monitor independently observes pins; a scoreboard independently checks outcomes.

Example: ina=2'b11 (3), inb=2'b10 (2): the mathematical expected output is 3'b101 (5). Use waveforms to confirm the precise serial output sampling order and en_o qualification in this implementation.

6. Monitors and prediction

A UVM monitor converts interface activity back into transaction objects and publishes them through analysis ports. This project has explicit input/expected and output/actual monitoring. Every published sample should own a separate object or immutable clone so later updates cannot alter an item already queued downstream.

sa_monitor.sv

Read this source alongside the RTL timing. Check transaction initialization, sampling cycles, DPI argument widths and which side generates the expected result.

Complete source: sa_monitor.sv (97 lines)

```
// Independent agents: input monitor reports expected transactions;
// output monitor reconstructs serial DUT results.
import "DPI-C" function chandle cl_predict_new(string name);
import "DPI-C" function int cl_predict_set_values_handle(int x,int y,chandle handle);

class sa_input_monitor extends uvm_monitor;
  `uvm_component_utils(sa_input_monitor)
  virtual sa_if m_vif;
  clk_data_config mon_config;
  uvm_analysis_port #(sa_transaction) input_ap;
  chandle hPredict;
  sa_transaction coverage_tx;
  covergroup sa_cg;
    ina_cp: coverpoint coverage_tx.ina;
    inb_cp: coverpoint coverage_tx.inb;
    ab_cross: cross ina_cp,inb_cp;
  endgroup
  function new(string name,uvm_component parent);
    super.new(name,parent);
    input_ap = new("input_ap",this);
    sa_cg = new();
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(clk_data_config)::get(this,"","config",mon_config))
      `uvm_fatal("INPUT_CFG","Missing clk_data_config")
    m_vif = mon_config.m_vif;
    if (m_vif == null) `uvm_fatal("INPUT_VIF","Null virtual interface")
    hPredict = cl_predict_new("serial_adder_predictor");
    if (hPredict == null) `uvm_fatal("DPI","Failed to allocate predictor")
  endfunction
  task run_phase(uvm_phase phase);
    sa_transaction tx;
    int state = 0;
    forever begin
      // Inputs were driven on negedge and are stable at this posedge.
      @(posedge m_vif.sig_clock);
      if (state == 0 && m_vif.sig_en_i) begin
        tx = sa_transaction::type_id::create("input_tx");
        tx.ina = {1'b0,m_vif.sig_ina};
        tx.inb = {1'b0,m_vif.sig_inb};
        state = 1;
      end else if (state == 1) begin
        tx.ina = {tx.ina[0],m_vif.sig_ina};
        tx.inb = {tx.inb[0],m_vif.sig_inb};
        tx.out = cl_predict_set_values_handle(int'(tx.ina),int'(tx.inb),hPredict);
        coverage_tx = tx;
        if (mon_config.coverage_enable) sa_cg.sample();
        // One unique transaction object is allocated per transfer.
        input_ap.write(tx);
        `uvm_info("INPUT_MON",tx.convert2string(),UVM_MEDIUM)
        tx = null;
        state = 0;
```

```
      end
    end
  endtask
endclass

class sa_output_monitor extends uvm_monitor;
  `uvm_component_utils(sa_output_monitor)
  virtual sa_if m_vif;
  clk_data_config mon_config;
  uvm_analysis_port #(sa_transaction) output_ap;
  function new(string name,uvm_component parent);
    super.new(name,parent);
    output_ap = new("output_ap",this);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(clk_data_config)::get(this,"","config",mon_config))
      `uvm_fatal("OUTPUT_CFG","Missing clk_data_config")
    m_vif = mon_config.m_vif;
    if (m_vif == null) `uvm_fatal("OUTPUT_VIF","Null virtual interface")
  endfunction
  task run_phase(uvm_phase phase);
    sa_transaction tx;
    int state = 0;
    forever begin
      // Sample at negedge: RTL posedge nonblocking updates are settled.
      @(negedge m_vif.sig_clock);
      if (state == 0 && m_vif.sig_en_o) begin
        tx = sa_transaction::type_id::create("output_tx");
        tx.out = {m_vif.sig_out,2'b0};
        state = 1;
      end else if (state == 1) begin
        tx.out[1] = m_vif.sig_out;
        state = 2;
      end else if (state == 2) begin
        tx.out[0] = m_vif.sig_out;
        output_ap.write(tx);
        `uvm_info("OUTPUT_MON",tx.convert2string(),UVM_MEDIUM)
        tx = null;
        state = 0;
      end
    end
  endtask
endclass
```

predict_class.cpp

Read this source alongside the RTL timing. Check transaction initialization, sampling cycles, DPI argument widths and which side generates the expected result.

Complete source: predict_class.cpp (34 lines)

```
#include "vpi_user.h"       // use vpi_printf so all output goes to transcript
#include "predict.hpp"
#include <string>
using namespace std;

class   CL_PREDICT {
  int ina, inb;
  const string m_name;
public:
    CL_PREDICT(string name) : m_name(name) {}
    void  set_values (int,int);
    int   do_predict () {return ina+inb;}
    const char* name () {return m_name.c_str();}
};

void CL_PREDICT::set_values (int x, int y) {
    ina   = x;
    inb   = y;
}
```

```
void* cl_predict_new(const char* name) {
    return (void*)( new CL_PREDICT(name) );
}

int cl_predict_set_values_handle(int x, int y, void* handle) {
    CL_PREDICT* r = static_cast<CL_PREDICT *>(handle);
    r->set_values(x,y);
    int out = r->do_predict();
    //vpi_printf("%s c++ predict: %d\n", r->name(), out);
    return out;
}
```

predict.hpp

Read this source alongside the RTL timing. Check transaction initialization, sampling cycles, DPI argument widths and which side generates the expected result.

Complete source: predict.hpp (163 lines)

```
/* MTI_DPI */

/*
 * Copyright 2002-2016 Mentor Graphics Corporation.
 *
 * Note:
 *   This file is automatically generated.
 *   Please do not edit this file - you will lose your edits.
 *
 * Settings when this file was generated:
 *   PLATFORM = 'win64'
 */
#ifndef INCLUDED_PREDICT
#define INCLUDED_PREDICT

#ifdef __cplusplus
#define DPI_LINK_DECL  extern "C"
#else
#define DPI_LINK_DECL
#endif

#include "svdpi.h"


#ifndef MTI_INCLUDED_TYPEDEF_uvm_hdl_path_slice
#define MTI_INCLUDED_TYPEDEF_uvm_hdl_path_slice

typedef struct {
    const char* path;
    int offset;
    int size;
}  uvm_hdl_path_slice;

#endif

#ifndef MTI_INCLUDED_TYPEDEF_uvm_printer_row_info
#define MTI_INCLUDED_TYPEDEF_uvm_printer_row_info

typedef struct {
    int level;
    const char* name;
    const char* type_name;
    const char* size;
    const char* val;
}  uvm_printer_row_info;

#endif
```

```
#ifndef MTI_INCLUDED_TYPEDEF_uvm_reg_bus_op
#define MTI_INCLUDED_TYPEDEF_uvm_reg_bus_op

typedef struct {
    int kind;
    svBitVecVal addr[SV_PACKED_DATA_NELEMS(64)];
    svBitVecVal data[SV_PACKED_DATA_NELEMS(64)];
    int n_bits;
    svBitVecVal byte_en[SV_PACKED_DATA_NELEMS(8)];
    int status;
}  uvm_reg_bus_op;

#endif


DPI_LINK_DECL DPI_DLLESPEC
void*
cl_predict_new(
    const char* name);

DPI_LINK_DECL DPI_DLLESPEC
int
cl_predict_set_values_handle(
    int x,
    int y,
    void* handle);

DPI_LINK_DECL DPI_DLLESPEC
const char*
uvm_dpi_get_next_arg_c(
    int init);

DPI_LINK_DECL DPI_DLLESPEC
const char*
uvm_dpi_get_tool_name_c();

DPI_LINK_DECL DPI_DLLESPEC
const char*
uvm_dpi_get_tool_version_c();

DPI_LINK_DECL DPI_DLLESPEC
void*
uvm_dpi_regcomp(
    const char* regex);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_dpi_regexec(
    void* preg,
    const char* str);

DPI_LINK_DECL DPI_DLLESPEC
void
uvm_dpi_regfree(
    void* preg);

DPI_LINK_DECL DPI_DLLESPEC
void
uvm_dump_re_cache();

DPI_LINK_DECL DPI_DLLESPEC
const char*
uvm_glob_to_re(
    const char* glob);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_hdl_check_path(
    const char* path);
```

```
DPI_LINK_DECL DPI_DLLESPEC
int
uvm_hdl_deposit(
    const char* path,
    const svLogicVecVal* value);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_hdl_force(
    const char* path,
    const svLogicVecVal* value);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_hdl_read(
    const char* path,
    svLogicVecVal* value);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_hdl_release(
    const char* path);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_hdl_release_and_read(
    const char* path,
    svLogicVecVal* value);

DPI_LINK_DECL DPI_DLLESPEC
int
uvm_re_match(
    const char* re,
    const char* str);

DPI_LINK_DECL void
m__uvm_report_dpi(
    int severity,
    const char* id,
    const char* message,
    int verbosity,
    const char* filename,
    int line);

#endif
```

7. Scoreboard and test control

The scoreboard receives expected and actual transactions through analysis exports feeding two analysis FIFOs. run_phase retrieves one item from each FIFO, compares the out fields with 4-state case equality, emits uvm_info on pass or uvm_error on mismatch, and increments counters. check_phase identifies missing comparisons and FIFO leftovers; report_phase prints counts.

The test raises a run-phase objection, starts the sequence on its sequencer, then allows output latency to drain before dropping the objection. A robust regression should explicitly check that the number of comparisons equals the number of stimulated transactions, and that zero failures occurred.

7.1 Where expected and actual transactions come from

In this SA project, expected and actual are local handles in sa_scoreboard.run_phase(). They are not connected directly to monitors. The connections are UVM TLM analysis ports and exports, established during connect_phase(); the handles receive queued objects when get() returns.

7.1.1 Two real source paths

EXPECTED: sa_input_monitor observes the serial input pins, reconstructs ina and inb, and calls the C++ DPI predictor to assign tx.out. It publishes that complete transaction using input_ap.write(tx).

ACTUAL: sa_output_monitor observes the serial DUT result (sig_out, qualified by sig_en_o), reconstructs tx.out, and calls output_ap.write(tx). Its transaction does not independently measure the input operands.

```
EXPECTED  input_mon.input_ap → sa_agent.expected_ap → sa_sb.expected_export →
expected_fifo.analysis_export → expected_fifo.get(expected)
ACTUAL    output_mon.output_ap → sa_agent.actual_ap → sa_sb.actual_export →
actual_fifo.analysis_export → actual_fifo.get(actual)
```

7.1.2 Connections made in sa_agent.connect_phase()

Exact source listing — sa_agent.sv

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  input_mon.input_ap.connect(expected_ap);
  output_mon.output_ap.connect(actual_ap);
  if (sa_drvr!=null) sa_drvr.seq_item_port.connect(sa_seqr.seq_item_export);
endfunction
```

The agent connects input_mon.input_ap to expected_ap and output_mon.output_ap to actual_ap. The final connection links sa_drvr.seq_item_port to sa_seqr.seq_item_export when the driver exists; it is the stimulus path, not a scoreboard result path.

7.1.3 Connections made in sa_env.connect_phase()

Exact source listing — sa_env.sv

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  sa_agent.expected_ap.connect(sa_sb.expected_export);
  sa_agent.actual_ap.connect(sa_sb.actual_export);
endfunction
```

Inside sa_env, sa_agent and sa_sb are instance handles. Each agent analysis port is connected to the corresponding uvm_analysis_export on sa_sb; no new prediction is calculated here.

7.1.4 Connections made in sa_scoreboard.connect_phase()

Exact source listing — sa_scoreboard.sv

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  expected_export.connect(expected_fifo.analysis_export);
  actual_export.connect(actual_fifo.analysis_export);
endfunction
```

In sa_scoreboard, expected_export and actual_export forward incoming analysis writes into the respective FIFO analysis_export. Both expected_fifo and actual_fifo are uvm_tlm_analysis_fifo #(sa_transaction) objects created in build_phase().

7.1.5 How run_phase() retrieves the transactions

Exact source listing — sa_scoreboard.sv

```
task run_phase(uvm_phase phase);
  sa_transaction expected,actual,actual_normalized;
  uvm_comparer comparer;
  forever begin
    expected_fifo.get(expected);
    actual_fifo.get(actual);
    // Output monitor observes only the serial sum. Supply the associated
    // input operands from the expected transaction so compare() checks
    // all three sa_transaction fields without false input mismatches.
    actual_normalized = sa_transaction::type_id::create("actual_normalized");
    actual_normalized.copy(actual);
    actual_normalized.ina = expected.ina;
    actual_normalized.inb = expected.inb;
    comparer = new();
    if (expected.compare(actual_normalized, comparer)) begin
      pass_count++;
```

```
        `uvm_info("ADDER_PASS",$sformatf("A=%0d B=%0d expected=%0d
actual=%0d",expected.ina,expected.inb,expected.out,actual.out),UVM_LOW)
      end else begin
        fail_count++;
        `uvm_error("ADDER_MISMATCH",$sformatf("A=%0d B=%0d expected=%0d
actual=%0d",expected.ina,expected.inb,expected.out,actual.out))
      end
    end
  endtask
```

expected_fifo.get(expected) and actual_fifo.get(actual) are blocking FIFO reads, not connect() calls. The expected item carries the observed operands and DPI-predicted out. The actual item carries the serial DUT output reconstructed by the output monitor. actual_normalized.copy(actual) makes a separate transaction, after which ina/inb are supplied from expected because the output monitor does not observe them. compare() uses the sa_transaction do_compare() hook; ADDER_PASS or ADDER_MISMATCH records the outcome and pass_count/fail_count is incremented. FIFO order must pair corresponding requests; this code has no transaction-ID matcher.

7.1.6 Worked example: ina=2 and inb=1

Input monitor: ina=2, inb=1; the DPI predictor calculates out=3. The expected FIFO receives {ina:2, inb:1, out:3}. Output monitor: reconstructed out=3; its unobserved operand fields retain their default values, typically zero. The actual FIFO receives {ina:0, inb:0, out:3}.

The scoreboard creates actual_normalized, copies actual, and sets its ina and inb to the expected operands. A three-field do_compare() then compares expected {2,1,3} with actual_normalized {2,1,3}. This does NOT prove the output monitor observed operands; it avoids false mismatches for fields outside that monitor's visibility.

7.1.7 Meaning of write(), connect(), and get()

write(tx): the monitor publishes a completed transaction on its analysis port. connect(): build-time UVM TLM connectivity routes published transactions to downstream ports, exports, and FIFO analysis_exports. get(tx): the scoreboard waits for and removes the next queued transaction; it assigns that object handle to its local variable.

Design caution: the simpler production check for an output-only monitor is expected.out versus actual.out, or a dedicated output transaction that compares only observed signals. Normalization is useful for demonstrating copy() and compare(), but FIFO-order correspondence and output sampling timing must still be verified.

sa_scoreboard.sv

Inspect the phase behavior carefully: run_phase is automatically scheduled by UVM, whereas an unrelated task run() is not.

Complete source: sa_scoreboard.sv (54 lines)

```
class sa_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(sa_scoreboard)
  uvm_analysis_export #(sa_transaction) expected_export;
  uvm_analysis_export #(sa_transaction) actual_export;
  uvm_tlm_analysis_fifo #(sa_transaction) expected_fifo;
  uvm_tlm_analysis_fifo #(sa_transaction) actual_fifo;
  int unsigned pass_count, fail_count;
  function new(string name,uvm_component parent);super.new(name,parent);endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    expected_export = new("expected_export",this);
    actual_export = new("actual_export",this);
    expected_fifo = new("expected_fifo",this);
    actual_fifo = new("actual_fifo",this);
  endfunction
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    expected_export.connect(expected_fifo.analysis_export);
    actual_export.connect(actual_fifo.analysis_export);
  endfunction
  task run_phase(uvm_phase phase);
```

```
    sa_transaction expected,actual,actual_normalized;
    uvm_comparer comparer;
    forever begin
      expected_fifo.get(expected);
      actual_fifo.get(actual);
      // Output monitor observes only the serial sum. Supply the associated
      // input operands from the expected transaction so compare() checks
      // all three sa_transaction fields without false input mismatches.
      actual_normalized = sa_transaction::type_id::create("actual_normalized");
      actual_normalized.copy(actual);
      actual_normalized.ina = expected.ina;
      actual_normalized.inb = expected.inb;
      comparer = new();
      if (expected.compare(actual_normalized, comparer)) begin
        pass_count++;
        `uvm_info("ADDER_PASS",$sformatf("A=%0d B=%0d expected=%0d
actual=%0d",expected.ina,expected.inb,expected.out,actual.out),UVM_LOW)
      end else begin
        fail_count++;
        `uvm_error("ADDER_MISMATCH",$sformatf("A=%0d B=%0d expected=%0d
actual=%0d",expected.ina,expected.inb,expected.out,actual.out))
      end
    end
  endtask
  function void check_phase(uvm_phase phase);
    super.check_phase(phase);
    if (pass_count+fail_count==0) `uvm_error("NO_CHECKS","No transactions compared")
    if (expected_fifo.used()!=0 || actual_fifo.used()!=0)
      `uvm_error("UNMATCHED",$sformatf("Remaining expected=%0d
actual=%0d",expected_fifo.used(),actual_fifo.used()))
  endfunction
  function void report_phase(uvm_phase phase);
    super.report_phase(phase);
    `uvm_info("ADDER_SUMMARY",$sformatf("PASSED=%0d FAILED=%0d",pass_count,fail_count),UVM_NONE)
  endfunction
endclass
```

sa_test.sv

Inspect the phase behavior carefully: run_phase is automatically scheduled by UVM, whereas an unrelated task run() is not.

Complete source: sa_test.sv (75 lines)

```
class sa_test extends uvm_test;
  `uvm_component_utils(sa_test)
  sa_env sa_env;
  function new(string name,uvm_component parent);super.new(name,parent);endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sa_env=sa_env::type_id::create("sa_env",this);
  endfunction
  // This method is run before the DUT sequence and exercises every requested
  // uvm_object hook through its standard PUBLIC API.
  task exercise_transaction_methods();
    sa_transaction original, copied, decoded, changed;
    uvm_comparer cmp;
    uvm_recorder rec;
    bit packed_bits[];
    int nbits;

    original = sa_transaction::type_id::create("original");
    copied   = sa_transaction::type_id::create("copied");
    decoded  = sa_transaction::type_id::create("decoded");
    changed  = sa_transaction::type_id::create("changed");
    original.ina = 2'd2;
    original.inb = 2'd3;
    original.out = 3'd5;

    // 1. convert2string()
    `uvm_info("SA_METHOD_STRING", original.convert2string(), UVM_LOW)
```

```
    // 2. do_copy() through copy()
    copied.copy(original);
    // 3. do_compare() through compare()
    cmp = new();
    if (!original.compare(copied, cmp))
      `uvm_fatal("SA_METHOD_COPY", "copy/compare failed")

    // 4. do_print() through sprint()/print()
    `uvm_info("SA_METHOD_PRINT", original.sprint(), UVM_LOW)

    // 5. do_record() through record() with a recorder.
    // Backend-specific waveform/database output needs the simulator's
    // transaction-recording configuration.
    rec = new("sa_method_recorder");
    original.record(rec);
    `uvm_info("SA_METHOD_RECORD", "Called record(rec): do_record executed", UVM_LOW)

    // 6. do_pack() and 7. do_unpack() through the bit-array APIs.
    nbits = original.pack_bits(packed_bits);
    if (nbits != 7)
      `uvm_fatal("SA_METHOD_PACK", $sformatf("Expected 7 packed bits; got %0d", nbits))
    void'(decoded.unpack_bits(packed_bits));
    cmp = new();
    if (!original.compare(decoded, cmp))
      `uvm_fatal("SA_METHOD_UNPACK", "pack/unpack round-trip failed")

    // Negative compare demonstrates field-by-field mismatch diagnostics.
    changed.copy(original);
    changed.out = original.out ^ 3'b001;
    cmp = new();
    cmp.show_max = 0; // Suppress intentional mismatch diagnostic output.
    if (original.compare(changed, cmp))
      `uvm_fatal("SA_METHOD_NEGATIVE", "Deliberately changed output compared equal")
    `uvm_info("SA_METHODS_PASS", "All seven UVM transaction methods exercised", UVM_NONE)
  endtask

  task run_phase(uvm_phase phase);
    sa_sequence seq;
    phase.raise_objection(this);
    exercise_transaction_methods();
    seq=sa_sequence::type_id::create("seq");
    seq.start(sa_env.sa_agent.sa_seqr);
    // Wait until the final three-bit output can be observed and compared.
    repeat (8) @(posedge sa_env.sa_agent.agent_config.m_vif.sig_clock);
    phase.drop_objection(this);
  endtask
endclass
```

8. Configuration, factory and environment

UVM constructs components in build_phase, obtains configuration using uvm_config_db, and connects TLM ports in connect_phase. The virtual interface bridges class-based testbench code to the static sa_if instance in the top-level module. The factory enables derived sequences/drivers to replace defaults without hard-coding constructors.

sa_if.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: sa_if.sv (11 lines)

```
`ifndef __SA_IF_SV__
`define __SA_IF_SV__
interface sa_if;
 logic  sig_clock;
 logic  sig_ina;
 logic  sig_inb;
```

```
 logic  sig_en_i;
 logic  sig_out;
 logic  sig_en_o;
endinterface: sa_if
`endif /* __SA_IF_SV__ */
```

sa_config.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: sa_config.sv (15 lines)

```
class sa_config extends uvm_object;
  `uvm_object_utils(sa_config)
  virtual sa_if m_vif;
  uvm_active_passive_enum is_active_sa = UVM_ACTIVE;
  bit checks_enable_sa = 1;
  bit coverage_enable_sa = 1;
  function new(string name="sa_config"); super.new(name);endfunction
endclass
class clk_data_config extends uvm_object;
  `uvm_object_utils(clk_data_config)
  virtual sa_if m_vif;
  uvm_active_passive_enum is_active_sa=UVM_ACTIVE;
  bit coverage_enable=1;
  function new(string name="clk_data_config"); super.new(name);endfunction
endclass
```

clkndata_config.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: clkndata_config.sv (14 lines)

```
`ifndef CLKNDATA_CONFIG_SV
`define CLKNDATA_CONFIG_SV
class clkndata_config extends uvm_object;
  // Do not register config class with the factory
  virtual clkndata_if      vif;
  uvm_active_passive_enum  is_active = UVM_ACTIVE;
  bit                      coverage_enable;
  bit                      checks_enable;
  extern function new(string name = "");
endclass : clkndata_config
function clkndata_config::new(string name = "");
  super.new(name);
endfunction : new
`endif // CLKNDATA_CONFIG_SV
```

sa_agent.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: sa_agent.sv (41 lines)

```
class sa_agent extends uvm_agent;
  `uvm_component_utils(sa_agent)
  sa_config agent_config;
  sa_sequencer sa_seqr;
  sa_driver sa_drvr;
  sa_input_monitor input_mon;
  sa_output_monitor output_mon;
  uvm_analysis_port #(sa_transaction) expected_ap;
  uvm_analysis_port #(sa_transaction) actual_ap;
  function new(string name,uvm_component parent);
    super.new(name,parent);
    expected_ap=new("expected_ap",this);
    actual_ap=new("actual_ap",this);
  endfunction
  function void build_phase(uvm_phase phase);
```

```
    clk_data_config cfg;
    super.build_phase(phase);
    if (!uvm_config_db#(sa_config)::get(this,"","config",agent_config))
      `uvm_fatal("AGENT_CFG","Missing sa_config")
    if (agent_config.m_vif==null) `uvm_fatal("AGENT_VIF","Null vif")
    cfg=new("cfg");
    cfg.m_vif=agent_config.m_vif;
    cfg.is_active_sa=agent_config.is_active_sa;
    cfg.coverage_enable=agent_config.coverage_enable_sa;
    uvm_config_db#(clk_data_config)::set(this,"input_mon","config",cfg);
    uvm_config_db#(clk_data_config)::set(this,"output_mon","config",cfg);
    input_mon=sa_input_monitor::type_id::create("input_mon",this);
    output_mon=sa_output_monitor::type_id::create("output_mon",this);
    if (agent_config.is_active_sa==UVM_ACTIVE) begin
      uvm_config_db#(clk_data_config)::set(this,"sa_drvr","config",cfg);
      sa_seqr=sa_sequencer::type_id::create("sa_seqr",this);
      sa_drvr=sa_driver::type_id::create("sa_drvr",this);
    end
  endfunction
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    input_mon.input_ap.connect(expected_ap);
    output_mon.output_ap.connect(actual_ap);
    if (sa_drvr!=null) sa_drvr.seq_item_port.connect(sa_seqr.seq_item_export);
  endfunction
endclass
```

sa_env.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: sa_env.sv (20 lines)

```
class sa_env extends uvm_env;
  `uvm_component_utils(sa_env)
  sa_config env_config;
  sa_agent sa_agent;
  sa_scoreboard sa_sb;
  function new(string name,uvm_component parent);super.new(name,parent);endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(sa_config)::get(this,"","config",env_config))
      `uvm_fatal("ENV_CFG","Missing sa_config")
    uvm_config_db#(sa_config)::set(this,"sa_agent","config",env_config);
    sa_agent=sa_agent::type_id::create("sa_agent",this);
    sa_sb=sa_scoreboard::type_id::create("sa_sb",this);
  endfunction
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    sa_agent.expected_ap.connect(sa_sb.expected_export);
    sa_agent.actual_ap.connect(sa_sb.actual_export);
  endfunction
endclass
```

sa_pkg.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: sa_pkg.sv (16 lines)

```
`ifndef __SA_PKG_SV__
`define __SA_PKG_SV__
`include "uvm_macros.svh"
package sa_pkg;
 import uvm_pkg::*;
 `include "sa_config.sv"
 `include "sa_transaction.svh"
 `include "sa_sequencer.sv"
 `include "sa_monitor.sv"
 `include "sa_driver.sv"
 `include "sa_agent.sv"
```

```
 `include "sa_scoreboard.sv"
 `include "sa_env.sv"
 `include "sa_test.sv"
endpackage: sa_pkg
`endif /* __SA_PKG_SV__ */
```

sa_tb_top.sv

This is infrastructure source: trace each create/get/set/connect and identify where the interface handle is first bound.

Complete source: sa_tb_top.sv (22 lines)

```
`timescale 1ns/1ps
`include "sa_if.sv"
`include "sa_pkg.sv"
module sa_tb_top;
  import uvm_pkg::*;
  import sa_pkg::*;
  sa_if vif();
  sa dut(.clk(vif.sig_clock),.en_i(vif.sig_en_i),.ina(vif.sig_ina),
                  .inb(vif.sig_inb),.en_o(vif.sig_en_o),.result_o(vif.sig_out));
  initial begin
    sa_config cfg;
    cfg=sa_config::type_id::create("cfg");
    cfg.m_vif=vif;
    cfg.is_active_sa=UVM_ACTIVE;
    cfg.checks_enable_sa=1;
    cfg.coverage_enable_sa=1;
    uvm_config_db#(sa_config)::set(null,"uvm_test_top.sa_env","config",cfg);
    run_test("sa_test");
  end
  initial vif.sig_clock=0;
  always #5 vif.sig_clock=~vif.sig_clock;
endmodule
```

9. Compiling and debugging in Questa

Compile VHDL before the SystemVerilog top level. The supplied source includes simulator scripts and a DPI C++ predictor; make sure the C++ shared library is built and loaded with the correct architecture/toolchain before simulation.

```
set UVMF_HOME=C:\UVMF_2023.4
vlib work
vcom -2008 -work work sa.vhd
vlog -sv -work work +incdir+. sa_tb_top.sv
```

The commands above are illustrative; use the packaged compile_sa.do / sim_ver_sa.do as the canonical project-specific starting point. This is a conventional UVM project, not automatically a generated UVMF project.

compile_sa.do

Follow each command in order and check the expected compiled library and simulation top name.

Complete source: compile_sa.do (8 lines)

```
# Questa/ModelSim mixed-language compilation; run from UVM/ directory.
# Requires a Questa license supporting mixed-language VHDL + SystemVerilog.
if {![file exists work]} {vlib work}
# Compile the VHDL DUT first (VHDL-2008).
vcom -2008 -work work sa.vhd
# Compile the SystemVerilog/UVM testbench second.
vlog -sv -work work +incdir+. sa_tb_top.sv
# C++ DPI predictor requires a separate shared-library build.
```

sim_ver_sa.do

Follow each command in order and check the expected compiled library and simulation top name.

Complete source: sim_ver_sa.do (9 lines)

```
# Launch after building the DPI shared library and setting PREDICT_LIB.
do compile_sa.do
if {![info exists PREDICT_LIB]} {
  echo "ERROR: set PREDICT_LIB to the C++ DPI shared-library path (without extension)"
} else {
  vsim -c -sv_lib $PREDICT_LIB work.sa_tb_top +UVM_TESTNAME=sa_test
  run -all
  quit -sim
}
```

wave_dut.do

Follow each command in order and check the expected compiled library and simulation top name.

Complete source: wave_dut.do (2 lines)

```
add wave -divider "VIRTUAL IF"
add wave -position insertpoint sim:/sa_tb_top/vif/*
```

wave_transactions.do

Follow each command in order and check the expected compiled library and simulation top name.

Complete source: wave_transactions.do (3 lines)

```
# UVM transactions (paths depend on simulator/UVM configuration)
# Use Questa GUI transaction viewer to browse /uvm_root/uvm_test_top/sa_env/sa_agent
# Driver: sa_drvr; monitors: input_mon and output_mon
```